Assignment 2 - Software Architecture of a 15-Minute Quick Commerce Platform

Subject: Software Architectures

Student: Gadamsetty V Sai Manoj Kumar | BITS ID: 2025MT12100

Domain: Retail (Quick Commerce)

Current market references: Blinkit, Zepto, Swiggy Instamart, BigBasket Now, Flipkart Minutes

ASR 1 Utility Tree - Ultra-Low Fulfillment Time

Why architecturally significant: In quick commerce, speed is the product. If confirmation and assignment are slow, customers abandon the order and often switch apps immediately.

Market examples: Blinkit and Zepto publicly emphasize fast confirmation and rider assignment as key user-facing metrics.

UTILITY TREE PATH	CONCRETE SCENARIO	PRIORITY	ARCHITECTURAL IMPACT
Utility -> Performance -> Response Time	At p95, checkout-to-order-confirmation stays under 2 seconds for active metro zones.	**(H, H)**	Needs in-memory cart/pricing, local inventory lookup, and async non-critical tasks.
Utility -> Performance -> End-to-End Latency	Order is assigned to picker within 30 seconds and rider within 90 seconds in high-density zones.	**(H, H)**	Requires event-driven orchestration and real-time capacity signals.
Utility -> Performance -> Predictability	ETA shown to customer stays within +/- 3 minutes for at least 90% of delivered orders.	**(H, M)**	Demands dynamic ETA service and frequent rider-location updates.

Practical note: This ASR shapes the entire hot path, from cache strategy to orchestration style.

ASR 2 Utility Tree - Inventory Accuracy and Consistency

Why architecturally significant: Nothing breaks trust faster than paying for items that are not actually available. Inventory errors also increase refunds, support load, and dark-store waste.

Market examples: Instamart and BigBasket Now rely heavily on frequent stock sync for top-moving SKUs.

UTILITY TREE PATH	CONCRETE SCENARIO	PRIORITY	ARCHITECTURAL IMPACT
Utility -> Consistency -> Inventory Correctness	For top 2000 SKUs, platform stock differs from dark-store stock by less than 1%.	**(H, H)**	Needs inventory reservation, real-time event updates, and reconciliation jobs.
Utility -> Reliability -> Reservation Safety	No oversell when two customers buy last unit simultaneously.	**(H, H)**	Forces atomic reserve-confirm-release workflow.
Utility -> Recoverability -> Data Repair	Mismatch between WMS and platform is detected and corrected automatically within 5 minutes.	**(H, M)**	Requires continuous reconciliation and compensating transactions.

Practical note: Reservation correctness is treated as a core business rule, not a background data-quality task.

ASR 3 Utility Tree - Availability During Peak Demand

Why architecturally significant: Quick commerce traffic is bursty (evenings, rain, festivals). If core services fail during spikes, orders are lost in minutes and operations fall behind quickly.

Market examples: Weekend evenings, rain spikes, and festival campaigns are common stress windows across major quick-commerce apps.

UTILITY TREE PATH	CONCRETE SCENARIO	PRIORITY	ARCHITECTURAL IMPACT
Utility -> Availability -> Fault Tolerance	During one zone failure, 99% browse traffic and 95% checkout traffic continue without manual intervention.	**(H, H)**	Needs multi-zone deployment, stateless services, health-aware routing.
Utility -> Scalability -> Burst Handling	System absorbs 5x order spike for 20 minutes without order loss.	**(H, H)**	Needs queue buffering, autoscaling, and back-pressure controls.

UTILITY TREE PATH	CONCRETE SCENARIO	PRIORITY	ARCHITECTURAL IMPACT
Utility -> Graceful Degradation -> Core Flow First	Recommendations service can fail without impacting search, cart, and checkout.	**(H, M)**	Requires service isolation, timeouts, and feature flags.

Practical note: Graceful degradation keeps revenue-critical paths alive when non-critical modules are unhealthy.

Tactics for ASR 1 - Ultra-Low Fulfillment Time

Recommended Tactics

- **Edge caching:** cache catalog snippets and dark-store metadata near user.
- **In-memory hot path:** cart totals and delivery fee in Redis for fast checkout.
- **Async orchestration:** order, picking, and rider assignment via events.
- **Locality routing:** assign nearest dark store and nearest rider first.
- **SLA budgets:** strict timeout per downstream call to avoid cascading delays.

What, Tool, and How

What to cache: SKU price, stock snapshot, delivery slots, store prep-time profile.

Tools: Redis Cluster, Kafka, gRPC, OpenTelemetry traces, geospatial index.

How it works: Checkout validates the cart from in-memory stock snapshot, then publishes OrderPlaced. Fulfillment Orchestrator triggers picker assignment and rider assignment in parallel, instead of waiting for serial steps.

Assumption: Dark stores publish near-real-time capacity and prep-time updates.

Expected Result

Fast checkout confirmations and reduced dispatch delay for dense city zones.

Tactics for ASR 2 - Inventory Accuracy and Consistency

Recommended Tactics

- **Inventory reservation:** reserve stock at checkout before payment capture.
- **Atomic state change:** use optimistic locking/version checks per SKU.
- **Event sourcing style updates:** every stock movement emits immutable event.
- **Compensation:** auto-release reservations on payment or picking failure.
- **Reconciliation worker:** compare WMS and platform stock periodically.

What, Tool, and How

What to protect: available-to-promise quantity, reservation ledger, final sold quantity.

Tools: PostgreSQL with row-versioning, Kafka topics, Debezium CDC, Redis for read model.

How it works: Inventory Service runs ReserveStock as a transaction, publishes Reserved event, and updates the read cache. Reconciliation jobs compare CDC stream with WMS snapshots and repair drift using compensating updates.

Trade-off: slightly higher write complexity is accepted to avoid oversell and post-payment cancellations.

Expected Result

Oversell incidents significantly reduced and cancellation/refund rates lowered.

Tactics for ASR 3 - Peak Availability and Scalability

Recommended Tactics

- **Active-active deployment:** multiple zones with automatic failover.
- **Queue buffering:** absorb burst orders in Kafka/RabbitMQ.
- **Horizontal autoscaling:** HPA based on CPU, latency, and queue lag.
- **Bulkheads + circuit breakers:** isolate recommendation/promotions from core checkout.
- **Graceful degradation:** disable non-essential features under overload.

Security Implementations

- **OAuth 2.1 + short-lived JWT:** access token 15 min, refresh token rotation enabled.
- **mTLS for service-to-service calls:** only trusted workloads can call internal APIs.
- **PII encryption and tokenization:** phone, address, and payment data protected at rest and in logs.
- **WAF + rate limiting:** blocks bot traffic, credential stuffing, and abusive checkout retries.
- **Secrets management:** DB/API credentials stored in Vault or cloud KMS-backed secret manager.

What, Tool, and How

What to protect: browse, add-to-cart, checkout, payment, and order tracking flows.

Tools: Kubernetes, API Gateway rate limits, Kafka, Prometheus alerts, feature flags, WAF, Istio mTLS, Vault/KMS.

How it works: Traffic spikes trigger autoscaling and controlled admission. Overflow work is queued and processed by workers, while users still receive accurate status updates for accepted orders. In parallel, the gateway enforces token and rate policies, and internal calls are validated via mTLS identities.

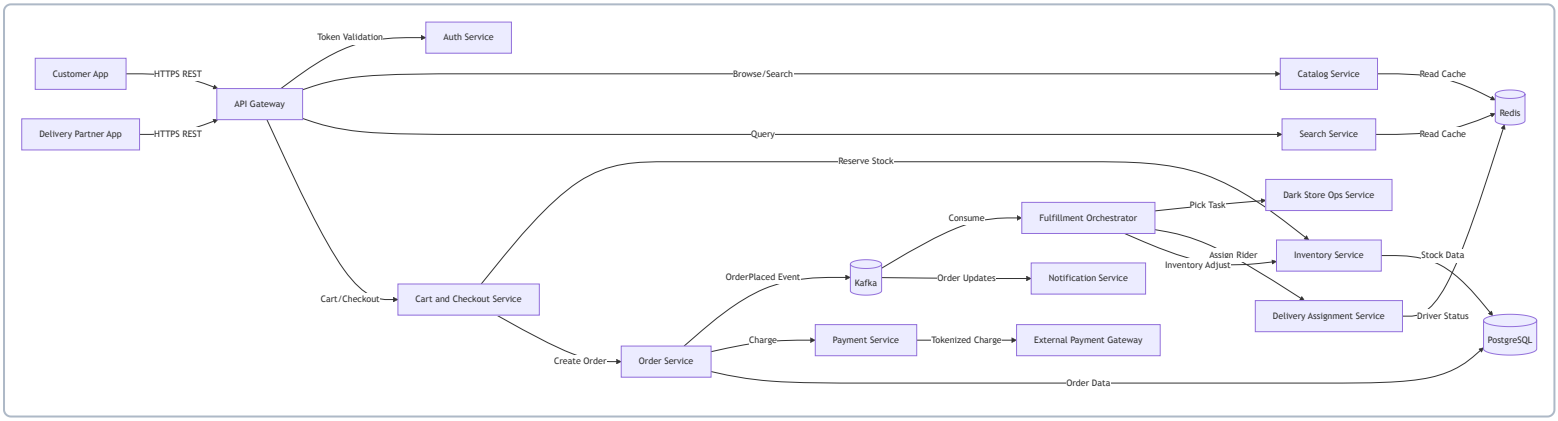
Trade-off: some non-critical features may be temporarily disabled to protect checkout reliability.

Expected Result

Platform remains usable under severe spikes with priority for core revenue path and stronger protection against abuse/fraud traffic.

Component and Connection View

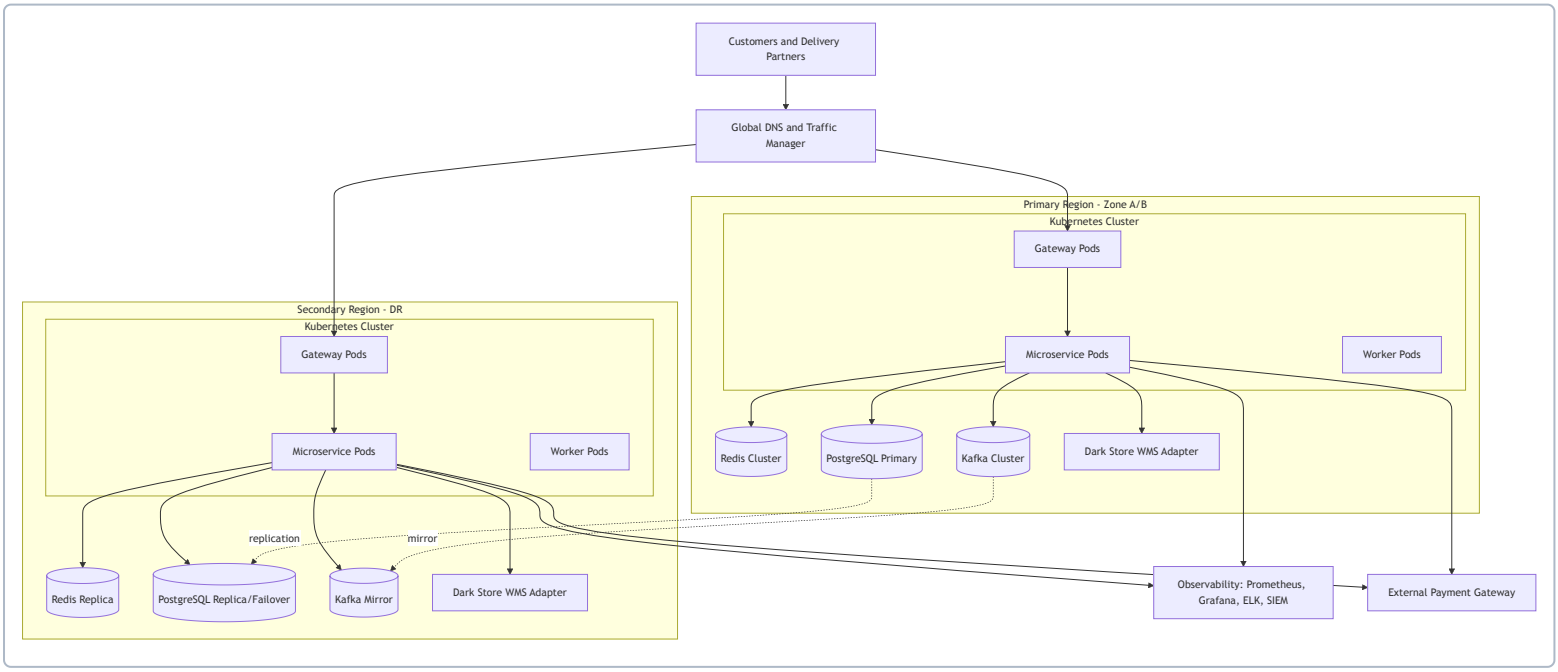
This view shows the runtime building blocks for ordering, inventory reservation, dark-store picking, rider assignment, and payment settlement.



Sync: HTTPS REST/gRPC | Async: Kafka events | Data: PostgreSQL + Redis

Deployment View

Deployment across primary and DR regions with zone redundancy, operational telemetry, and external dependency boundaries.



Deployment tactics shown: zone redundancy, autoscaling workers, buffered event processing, external dependency isolation.

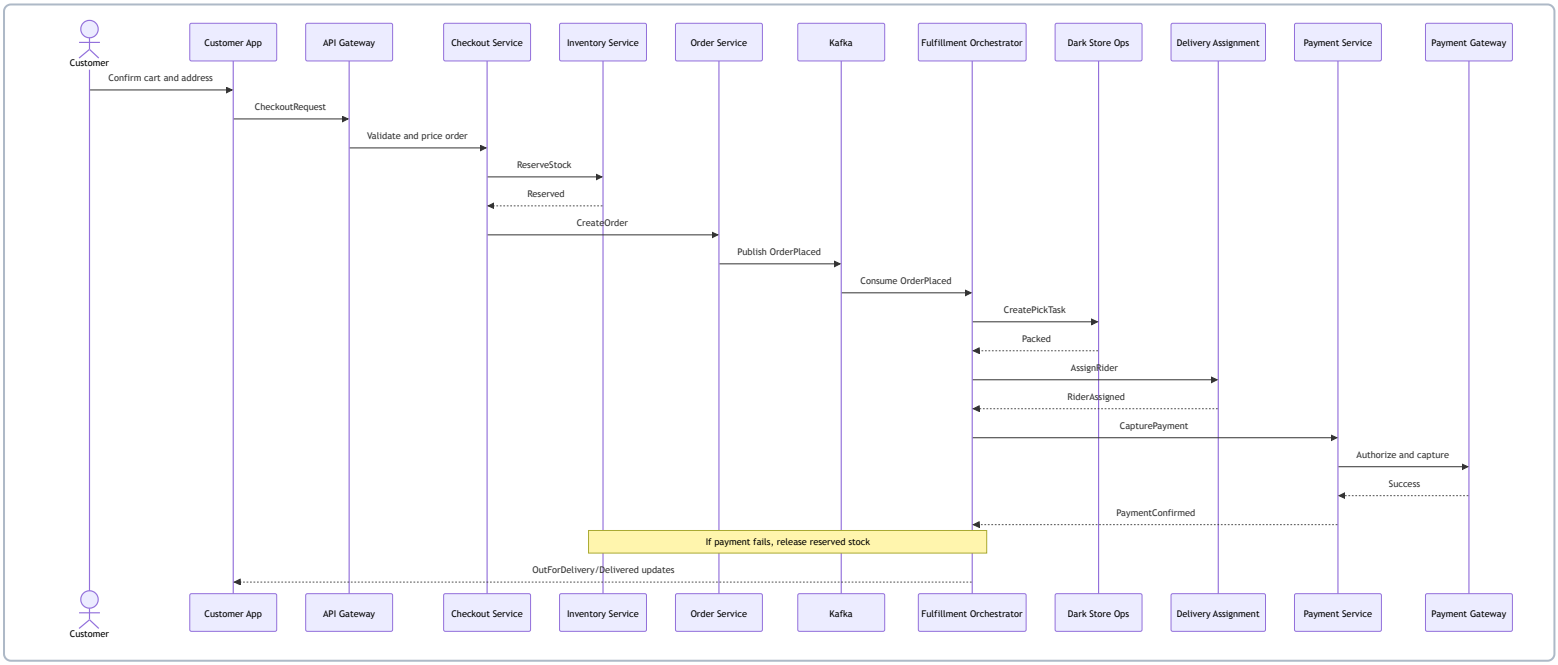
Security implementation in deployment: private subnets for data services, encryption-at-rest for Redis/PostgreSQL, workload identity for pods, centralized audit logs to SIEM, and restricted admin access through VPN/bastion.

Important Inter-Component Messages and Communication Methods

#	FROM -> TO	MESSAGE	COMMUNICATION METHOD	PURPOSE
1	Customer App -> API Gateway	CheckoutRequest(cartId, address, slot)	HTTPS REST/JSON	Start order placement
2	Checkout Service -> Inventory Service	ReserveStock(orderId, skuList)	gRPC unary	Prevent oversell before payment
3	Order Service -> Kafka	OrderPlaced event	Async publish (Kafka)	Trigger fulfillment pipeline
4	Fulfillment Orchestrator -> Dark Store Ops	CreatePickTask(orderId, priority)	gRPC unary	Start picking and packing workflow
5	Fulfillment Orchestrator -> Delivery Assignment	AssignRider(orderId, location)	gRPC streaming	Allocate nearest available rider
6	Order Service -> Payment Service	ChargeOrder(orderId, amount, paymentToken)	HTTPS REST + idempotency-key	Safe and retryable payment
7	Payment Service -> Payment Gateway	AuthorizeCapture(token, amount)	HTTPS REST + mTLS	External payment authorization and capture
8	Kafka -> Notification Service	OrderConfirmed/Picked/OutForDelivery/Delivered	Async consume	Push real-time user updates

Sequence Diagram - Major Scenario: Place and Deliver a Quick Commerce Order

Scenario: Customer places an order, stock is reserved, dark store packs items, rider delivers, and payment is finalized.



Architecture Pattern 1 - Microservices with Bounded Contexts

Where it is used

- Separate services for Catalog, Search, Cart, Inventory, Order, Fulfillment, Delivery, Payments.
- API Gateway provides single entry for mobile apps and handles auth/throttling.

Why this pattern fits

- Different scaling needs: Search and Checkout traffic are much higher than admin flows.

- Independent release: inventory and fulfillment can evolve without catalog redeploy.
- Better team autonomy and ownership per business function.

Trade-off handled

Distributed complexity is managed through service contracts, consumer-driven schema checks, and centralized observability dashboards.

Architecture Pattern 2 - Event-Driven Orchestration (Saga)

Where it is used

- Order lifecycle is modeled as events: OrderPlaced, StockReserved, Packed, RiderAssigned, Delivered.
- Orchestrator coordinates cross-service steps and compensation on failure.

Why this pattern fits

- High-throughput and loose coupling for spike handling.
- Natural fit for long-running workflows (reserve, pick, assign, deliver, settle).
- Improves auditability and operational replay/debugging.

Trade-off handled

Eventual consistency risk is reduced using idempotent consumers, retries with backoff, dead-letter queues, and compensation handlers.

Product note: This event-driven lifecycle is visible in products like Blinkit and Zepto where order-state updates are stream-oriented.

Key Learnings (3)

While preparing this architecture, these were my main takeaways:

1. **Business promise defines architecture:** a 10-20 minute delivery SLA forces low-latency design and parallel orchestration from day one.
2. **Inventory is the backbone of trust:** reservation and reconciliation logic must be treated as core architecture, not secondary logic.
3. **Design is only half the job:** clear message contracts, observability, and fallback behavior make the architecture practical in real operations.